

Spec Driven Development

Aplicaciones

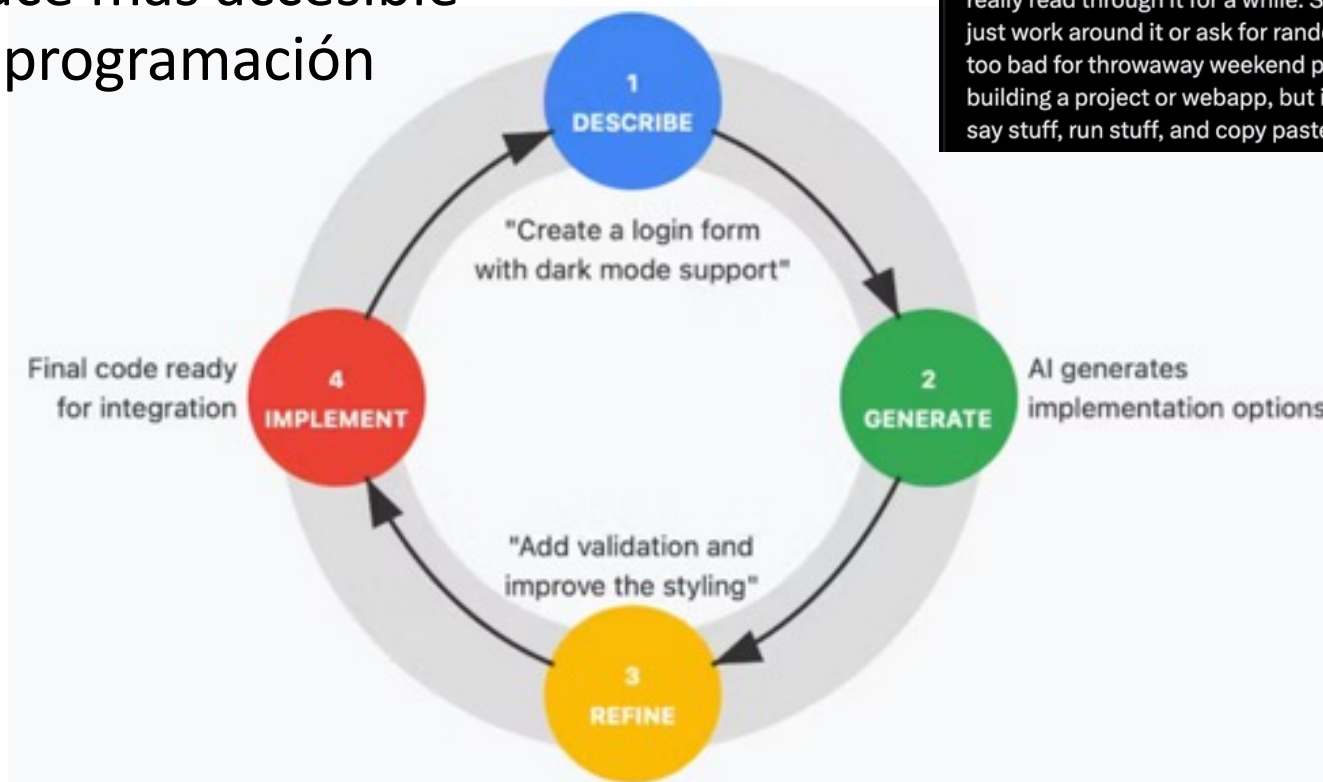
Agentes LLM para Código

- Sistemas basados en IA Generativa (LLMs) que asisten o incluso ejecutan de forma autónoma distintas actividades del ciclo de desarrollo de software (SDLC)
 - Más allá de autocompletar o proveer un chat
 - Puede estar integrados a un IDE o bien ser parte de una consola (CLI)



El inicio: Vibe coding

- Usar IA para crear código
- No tiene que ver tanto con saber programar, sino con saber **qué programar**
- Hace más accesible la programación



Algunos comentarios

- Tiene muchas ventajas para prototipar ideas, o para audiencias menos técnicas
- Pero ...
 - ¿Qué tamaño o calidad de sistemas puedo desarrollar?
 - ¿Colaboración entre miembros del equipo?
 - ¿Cómo mantengo el sistema?
 - ¿Qué pasa si la IA se equivoca?
 - ¿Quién control/revisa lo generado?



- **Proceso**

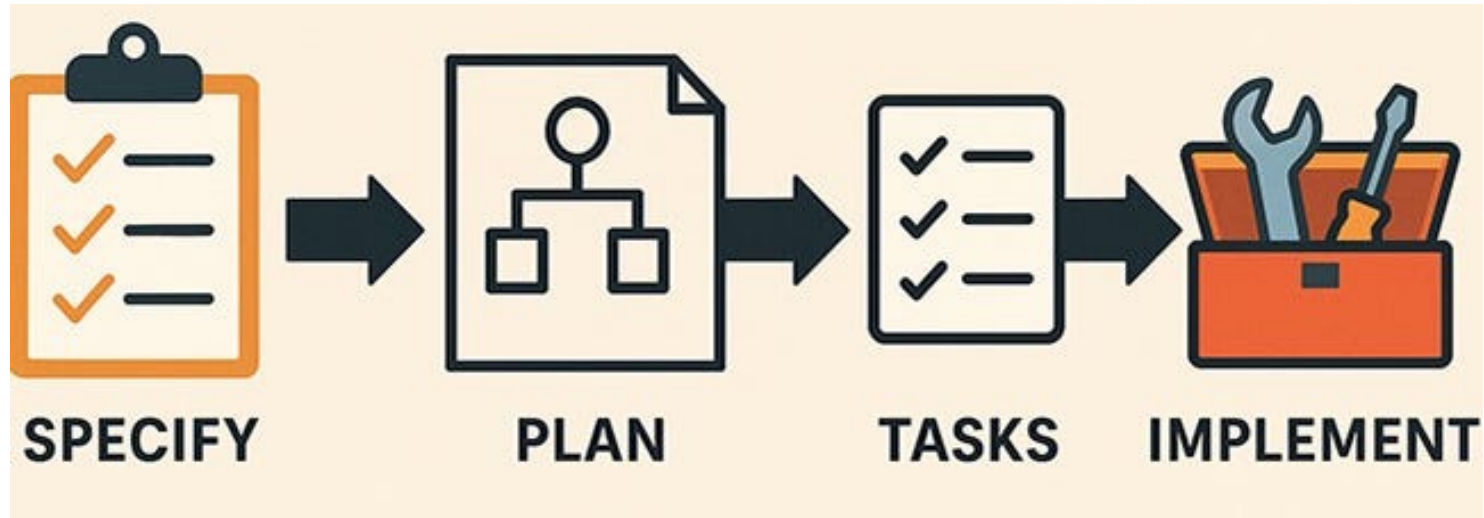


Spec Driven Development

- Surge como una visión más “ordenada” del vibe coding, que permite seguir un proceso repetible y le asegura un mayor control al desarrollador
- **Idea** (de los 70’): Escribir la especificación primero, y luego construir el sistema en base a ella
 - Lo “nuevo” es que los agentes de IA pueden participar de forma activa en cada etapa del ciclo de vida, lo cual hace el proceso más ágil y productivo
 - La especificación es un artefacto “vivo” (se define al principio, pero va evolucionando con el proyecto) y todo lo que se construye se va validando contra ella
 - Por ej., OpenAPI (Swagger) para servicios REST o modelos ER para bases de datos, pero llevados a nivel sistema y con mucho más lenguaje natural



Spec Driven Development



OpenSpec



Kiro



Agent OS



Etapas

1. Captura de requerimientos
2. Redacción de la especificación
3. Validación y refinamiento
4. Generación de código e implementación
5. Validación continua

- Mejores prácticas

- Proveer el **contexto** correcto para cada tarea
- Mantener un **contexto** general del proyecto
- **Versionar** el código, pero también las especificaciones
- No olvidarse de ir desarrollando **tests** en paralelo al desarrollo
- Mantener la documentación actualizada con el código



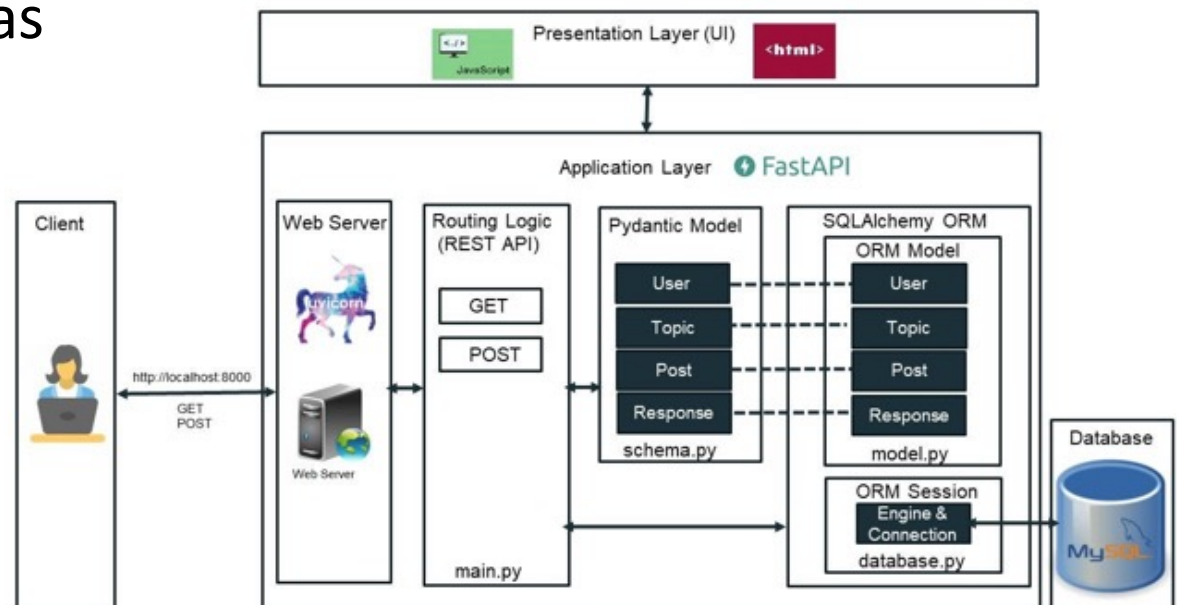
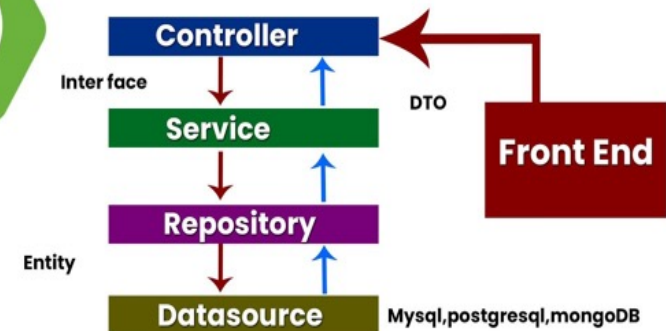
Relación con Arq. Web

1. Contexto, requerimientos
2. Definición de entidades
3. Identificación de servicios
4. Arquitectura monolítica o de microservicios
5. Preferencia de lenguaje y tecnologías
6. Seguridad (ej., JWT)
7. ...

Spring Boot



Layered Architecture



Possible SDD: Gemini CLI + Conductor

<https://github.com/google-gemini/gemini-cli>

`npm install -g @google/gemini-cli`



<https://github.com/gemini-cli-extensions/conductor>

`gemini extensions install https://github.com/gemini-cli-extensions/conductor --auto-update`

```
# Project: My TypeScript Library

## General Instructions

- When you generate new TypeScript code, follow the existing coding style.
- Ensure all new functions and classes have JSDoc comments.
- Prefer functional programming paradigms where appropriate.

## Coding Style

- Use 2 spaces for indentation.
- Prefix interface names with 'I' (for example, 'IUserService').
- Always use strict equality ('===' and '!==').
```

```
> tree .
.
├── GEMINI.md
├── test2
│   ├── GEMINI.md
│   └── index.html
└── test3
    ├── a
    │   └── GEMINI.md
    ├── b
    │   └── GEMINI.md
    ├── c
    │   └── GEMINI.md
    └── GEMINI.md

6 directories, 7 files

~/code/tmp/test3
>
```



Comandos de Conductor

Command	Description	Artifacts
<code>/conductor:setup</code>	Scaffolds the project and sets up the Conductor environment. Run this once per project.	<code>conductor/product.md</code> <code>conductor/product-guidelines.md</code> <code>conductor/tech-stack.md</code> <code>conductor/workflow.md</code> <code>conductor/tracks.md</code>
<code>/conductor:newTrack</code>	Starts a new feature or bug track. Generates <code>spec.md</code> and <code>plan.md</code> .	<code>conductor/tracks/<id>/spec.md</code> <code>conductor/tracks/<id>/plan.md</code> <code>conductor/tracks.md</code>
<code>/conductor:implement</code>	Executes the tasks defined in the current track's plan.	<code>conductor/tracks.md</code> <code>conductor/tracks/<id>/plan.md</code>
<code>/conductor:status</code>	Displays the current progress of the tracks file and active tracks.	Reads <code>conductor/tracks.md</code>
<code>/conductor:revert</code>	Reverts a track, phase, or task by analyzing git history.	Reverts git history
<code>/conductor:review</code>	Reviews completed work against guidelines and the plan.	Reads <code>plan.md</code> , <code>product-guidelines.md</code>



Hands-on

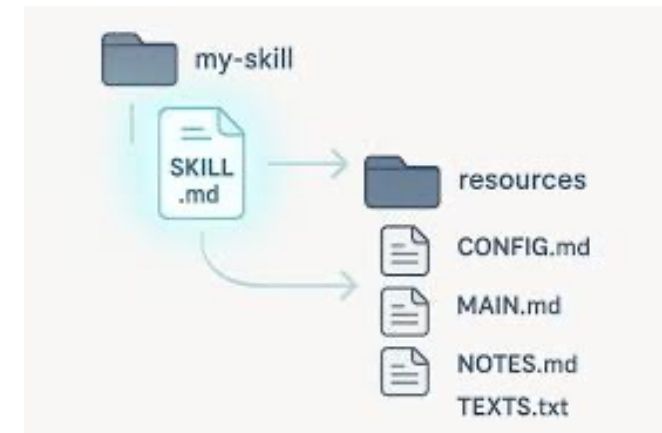


```
1 # Project Context
2
3 ## Definition
4 - [Product Definition](./product.md)
5 - [Product Guidelines](./product-guidelines.md)
6 - [Tech Stack](./tech-stack.md)
7
8 ## Workflow
9 - [Workflow](./workflow.md)
10 - [Code Style Guides](./code_styleguides/)
11
12 ## Management
13 - [Tracks Registry](./tracks.md)
14 - [Tracks Directory](./tracks/)
```



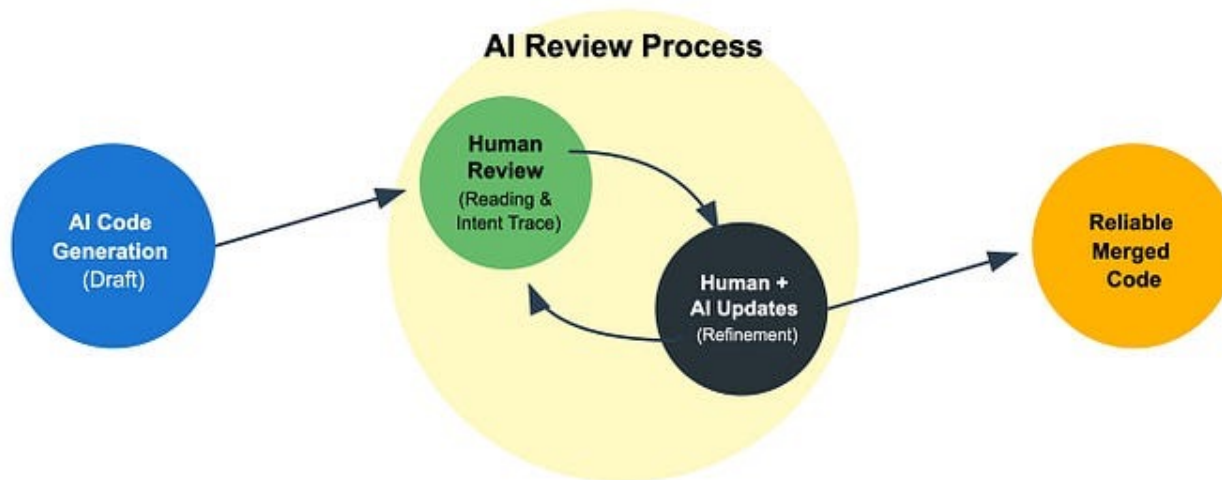
Skills

- Capacidades plug-and-play para un LLM (o agente), que le permiten realizar tareas concretas
 - Instrucciones precisas, tipo prompt
 - El agente las carga en su “contexto” de manera progresiva, dependiendo de la necesidad
 - Pueden referenciar y delegar en scripts, comandos, otras skills, etc.
- Se especifican en carpetas en formato Markdown, con un template estandarizado



Controlando la IA

TREAT AI-GENERATED CODE AS A DRAFT



**NEVER COMMIT
CODE YOU
CAN'T EXPLAIN
TO OTHERS**
OR EVEN YOURSELF



